

Dynamic Evolution of Business Performance Management

Liangzhao Zeng, Hui Lei, Michael Dikun, Henry Chang
IBM T.J. Watson Research Center
lzeng, hlei, mdikun, hychang@us.ibm.com

Chang Shu
IBM China Software Development Laboratory
shuchang@cn.ibm.com

Abstract

Business Performance Management(BPM) is a new frontier in IT-enabled enterprise that supports the monitoring business operations. BPM solutions must be able to efficiently process business events, compute business metrics, detect business situations, and provide the real-time visibility of key performance indicators (KPIs) in dynamic environments, wherein sources of dynamicity are plentiful, including new strategies, operations, KPIs, etc. Therefore, BPM solutions need to adapt these changes by dynamic evolution. We propose a policy-driven approach, where evolution policies capture the evolution mechanism of BPM solutions. This frees solution developers from low-level programming concerns. We implemented a hybrid compilation-interpretation framework that enables execution of wide spectrums of evolution policies. Further, our framework enables execution of evolution policies in parallel with on going event processing and guarantees the integrity on both of them.

1 Introduction

In order to function effectively in today's business environment, organizations must have visibility in their business activities and operation performance at all times. This allows them to stay competitive and profitable. BPM is a new generation of enterprise data management system that focuses on monitoring business operations. BPM solutions must be able to efficiently process business events, compute business metrics, detect business situations, and provide the real-time visibility of key performance indicators (KPIs). A BPM solution lifecycle involves five steps (see Figure 1).

Step 1. A BPM solution lifecycle starts with modeling a BPM solution. We define an *observation model* to be an abstraction of an observation system, which captures the re-

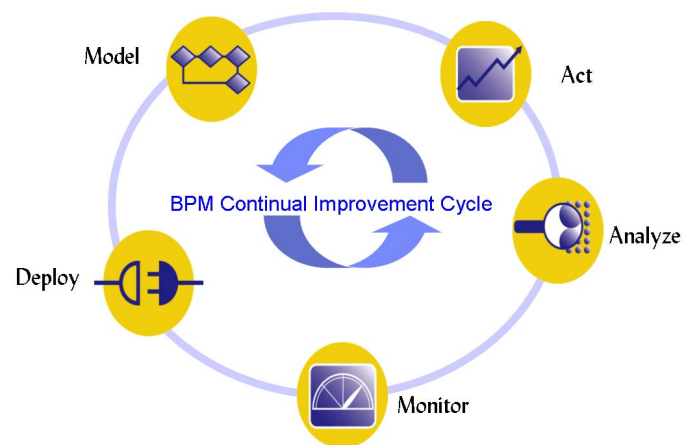


Figure 1. Business Performance Management Lifecycle

quirements of BPM solutions. In an observation model, a collection of metrics is defined to measure the performance of business operations, and a set of situation is declared to identify critical business conditions. **Step 2.** In the deployment phase, after performing a series of transformations on an observation model, the executable code is generated and then deployed into the runtime platform. **Step 3.** Once the BPM runtime component is deployed, it monitors business activities by measuring metrics and detecting business situations. **Step 4.** In the next phase, the monitoring results are used to perform further analysis, for example, what-if or root-cause analysis. **Step 5.** The final phase is taking actions for improving the business performance. The actions may include re-engineering business processes, etc.

The goal of each lifecycle is to improve the business performance, and by repeating lifecycles, the business performance is continually improved. In particular, when business performance improvement action is taken, one life-

cycle is completed and another lifecycle is started. In the new lifecycle, a new observation model needs to be developed. When the new BPM solution is deployed, the evolution from the existing BPM solution to the newer one is required.

A native evolution mechanism is hotswap, i.e., when the new solution is deployed, the existing solution is forced to shutdown. Such an approach is easy to implement, however, it is not sufficient for supporting continual BPM lifecycle. First, the hotswap evolution may lead to information loss. The BPM is an event-driven stateful system, wherein business situations are detected based on occurrences of events. When a solution is forced to shutdown manually, business situation detection is interrupted, which may eventually result in missing the situation. Second, the hotswap is not practical in some business scenarios. In some cases, the existing and new BPM solutions need to coexist for a period of time, during which the transition from the existing BPM solution to the newer one should be performed. For example, a BPM solution is developed and deployed to monitor a business process. Later on, based on BPM's monitoring results and analysis, an action that re-engineers business processes is taken. Therefore, the old process needs to be migrated to the new process. Accordingly, when a new BPM solution that monitors the new process is developed and deployed, the evolution of BPM solutions need to be synchronized with business process migration. Thus, other than hotswap, transition-based evolution is required.

Implementing dynamic evolution of the BPM solution that enables continual business performance improvement is by no means a trivial task. There are three major challenges:-

1. Complexity of Mapping Relationship among Solutions. There are large numbers of the business entities that need to be monitored, and these business entities may be associated with an even larger number of metrics. Further, the data structure for describing metrics may be a complex type. Therefore, when one BPM solution migrates to another, element (i.e., context, metric and situation) mapping relationship can be complicated. The challenge is to enable solution evolution without involving the solution designer's much programming efforts.

2. Integrity of Event Processing and Solution Evolution. The monitored business entities (e.g., business processes) operate continually, which requires the BPM runtime engine to process events continually. This means that new BPM solutions need to be deployed without shutting down the BPM runtime engine (i.e., hot deployment). When one BPM lifecycle transits to another, multiple BPM solutions may coexist. It is a big challenge for the BPM runtime engine to guarantee the integrity on both event processing (i.e., completion of ongoing event process transaction) and solution evolutions (i.e., completion of state informa-

tion migration).

3. Validation of Runtime State Information. When migrating the state information from one BPM solution to another, state information validation is required in the target solution. For example, when a context instance migrates to the target solution, the system needs to check whether the migrated context instance is a duplication of an existing context instance in the target solution. The difficulty is that state information in both solutions is not static, as the state information is updated when event processions are still ongoing.

In this project, we advocate a policy-driven approach to implement dynamic BPM solution evolutions. In general, a policy provides an abstraction of system behaviors that allow users to manage the system without needing to understand extraneous details. In our approach, evolution policies are used to capture the mechanism of solution evolution and executed systematically by the runtime infrastructure. Without system support, solution developers need to focus on developing a time-consuming platform dependent implementation. While in our policy-driven approach, solution providers can concentrate on business processes and observation models, without worrying about platform specific implementation details. Further, wide spectrums of evolution policies are supported. In a nutshell, the salient features and contributions of our work are:

1. Policy-Driven Solution Evolution. In order to capture the BPM solution evolution knowledge, we have designed an evolution policy language. The evolution policies define two aspects of a solution migration process: (i) how to sunset the existing solution transactionally; (ii) how to migrate the data from exiting solution to a newer one. In our approach, the evolution policies are declaratively defined by solution providers. It should be noted that these evolution policies are consumed by our runtime infrastructure to conduct the solution evolutions transparently to the users.

2. Hybrid Compilation-interpretation Framework. In our previous work, we proposed a hybrid compilation-interpretation framework to implement event processing, where observation models are decomposed into *common logic*, *information logic* and *expression logic*. The common logic is generic to any observation model, information logic describes elements in an observation model, and expression logic represents the computation relationship among the elements. During the transformation, the expression logic is compiled to an executable library. At runtime, the engine that implements the common logic interprets the information and loads the library to realize the monitoring capability. In this paper, we follow the same design principles and benefit from such a hybrid framework. We extend the runtime infrastructure to transform and interpret evolution policies, which allows fine grained controls on solution evo-

lution. By integrating with event processing logic, our approach can guarantee the integrity on both solution evolution and event processing. Further, such a framework supports execution of wide spectrums of evolution policies.

3. Model-driven Runtime State Validation. When migrating the data from an existing BPM solution to a new one, it is important to check whether the migrated data is valid in the new BPM solution. In our approach, during transformation time, we generate customize code for efficient runtime state validation. Also, we derive relationships among the event processing progress and runtime state based on the observation model. Such relationship information allows more efficient verification of the migrating data and parallelization of solution evolution and event processing.

Our work tackles the software evolution issue in model-driven architectures. It should be noted that that in our previous work [10], we mainly focused on how to realize a BPM solution lifecycle. However, in this paper, we address the issue of evolutions from one BPM solution lifecycle to another. The remainder of this paper is organized as follows: Section 2 introduces the important concepts of observation modeling. Section 3 discusses the definition of evolution policy. Section 4 presents the execution of evolution policies. Finally, we discuss some related work in Section 5 and provide concluding remarks in Section 6.

2 Observation Meta-Model

The requirements of a BPM solution are captured by an observation model. The specification of observation models is given by the *observation meta-model*. In this section, we use an example of observation model (see Figure 2) to illustrate the design of observation meta-model. The meta-model is a simplified version of [3]. The meta-model describes two aspects of an observation model.

2.1 Information organization

The information in an observation model is typically constructed top-down. Formally, an observation model contains a hierarchy of contexts. For example, in Figure 2, the root context contains two contexts *store* and *warehouse*, and context *store* further contains context *customer*. A context corresponds to an entity to be monitored. There are two kinds of context: statically defined (for example, *warehouse*) or dynamically created (for example, *items* in a *warehouse*). From an information organization point of view, context can be considered as a container for *metrics*. Metrics are the knowledge about business operation performance of the entity being observed. As an example, a metric can be the *stockLevel* of a *item* in *warehouse*. More specifically, *key metrics* are used

to identify a context instance. For example, *storeID* is the key metric that is used to identify context *store*. In same case, multiple metrics are defined as combination key for identifying context instances. *Situations* are a boolean type of metrics, and represent the gating conditions for generating *alters*. A context associates with a collection of events. Events report up-to-date status information about business operations, which are used to compute the value of metrics. It should be noted that the data structure for defining metrics and events can be primitive, structured, and single or multiple value type.

2.2 Operation of observation model

There are two aspects regarding the operation of an observation model: (1) How the events are being processed. An event is associated with a *filter predicate* and a *correlation predicate*. The filter predicates specify the types of events that need to be processed. Correlation predicates determine the associations between events and context instances. At runtime, an event is either correlated into existing context instances, or creates new context instance, or raise exception. It should be noted that a metric in the context is used to save the value of an event. (2) Metric network. We adopt a graphical model that captures the dependency relationships among metrics. The vertexes represent either metrics or dependencies among metrics. The edges represent the relationship between a dependency and a metric: either *inputslot* or *outputslot*. *Map expressions* are used to denote the association between a dependency vertex and the input/output metrics. As an example, in Figure 2, a map expression (map 3) is shown as:

```
item.stockLevel := minus(item.stockLevel,
customer.order.lineItem.amount)
```

In this expression, *minus* is a dependency vertex, and *item.stockLevel* and *customer.order.lineItem.amount* are metric vertexes. The *item.stockLevel* and *customer.order.lineItem.amount* are *inputslots* and *item.stockLevel* is an *outputslot* of dependency *minus*. The metrics in a map expression may belong to different contexts. For example, in the above expression, two metrics are from context *item* and *customer* respectively. The relationship supported by a metric network includes functional, probabilistic, system dynamics and extensible user-defined dependency. Currently, we focus on explicit functional dependency. It should be noted that the execution of an expression can be triggered by an incoming event, value changes of a metric, or occurrence of a situation.

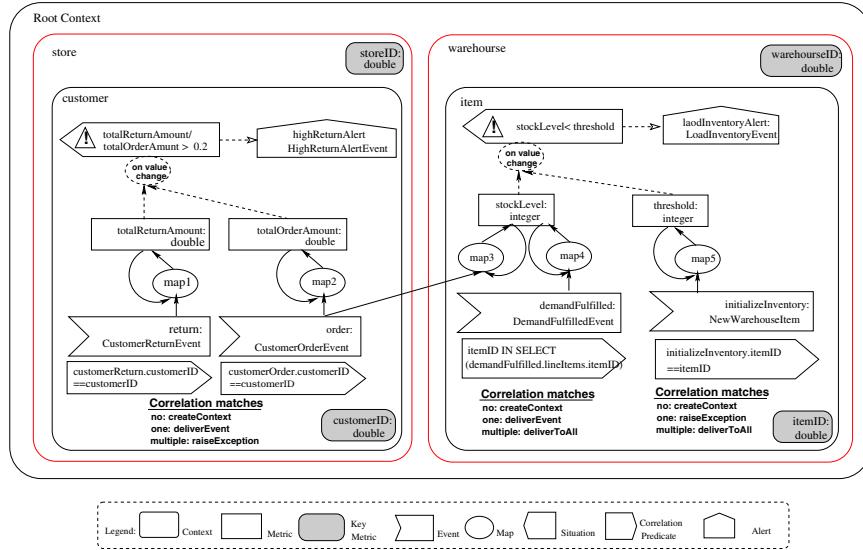


Figure 2. An Example of Observation Model

3 Evolution Policy

In order to realize the BPM evolution, we extend the Observation Meta-model with *Evolution Policy* to describe the process of BPM solution evolution. There are two aspects in the evolution process: (i) How to phase out the exiting model transactionally (sunset policy); (ii) How to migrate the context instances and associated elements (i.e., metrics and situations) from the existing solution to the newer one (context mapping policy).

3.1 Sunset Policy

A collection of sunset policies define the process of phasing out a BPM solution, in which a policy is denoted as $T[P] \mid A$. T indicates type of contexts or events in the observation model. P is a predicate for selecting either context or event instances. A is the action for managing the selected instances. There are two kinds of policies, namely *context instance termination policy* (if T is referred to a context) and *event handling policy* (if T is referred to an event).

1. Context instance termination policy. Each type of context in an observation model is associated with a collection of context instance termination policies. There are two different actions in this type of policies:

- **Terminate:** forces the selected context instances to be discarded. An example of termination policy can be

```
Item::item[item.stockLevel == 0] |
terminate(item, timestamp)
```

which means for monitoring context instance *item* (the context type is *Item*), if its metric

stockLevel's value equals to zero, then terminate the context instance *item* by timestamp.

- **Keep alive:** indicates that the selected context instances continue operating until terminated by the terminating situation that originally defined by the observation model. An example of a keep alive policy can be

```
Item::item[item.stockLevel > 0] |
keepAlive (item)
```

which means for monitor context instance *item* (the context type is *Item*), if its metric *stockLevel*'s value is greater than zero, then keep alive context instance *item*.

2. Event handling policy. Each type of event in an observation model is associated with a collection of event handling policies. There are also three types of actions for events:

- **Ignore:** indicates the event selected by the predicate is ignored.
- **Correlate with existing context instances only:** indicates that the selected events only correlate with existing context instance only, but not creating any new context instances.
- **Correlate with existing context instance or create new context instances:** indicates that selected events are used to correlate with existing context or create new context instances.

An example of an event handling policy can be

```
CustomerOrderEvent:: customerOrder[]|
ignore (customerOrder)
```

which indicates that the all the customerOrder events will be ignored.

3.2 Context Migration Policy

Context migration policy defines how context instances are migrated to a newer solution, which can be notated as: $C_s[P] \mid C_t(E)$, where C_s is the source context type in existing BPM solution, P is a predicate for selecting context instances, C_t is the target context type that selected context instances migrated to, and E is list of expressions that define element mapping relationship between the source and target context. In general, an expression can be denoted as:

$$V'.C.\lambda := \chi(V.C_1.\lambda_1, \dots, V.C_n.\lambda_n, V.\gamma_1, \dots, V.\gamma_m, V.\varepsilon_1, \dots, V.\varepsilon_k) \quad (1)$$

χ is the operator and there are three kinds of operands: metrics (λ_i), events (γ_i) and external data sources (ε_i)¹. An example of a migration policy can be

```
V.Customer[Customer.totalReturn > 0]|
( V'.Customer.ID:=V.Customer.ID;
V'.Customer.totalReturn:=V.Customer.totalReturn)
```

which means for a monitoring context instance customer (the context type is Customer), if its metric totalReturn's value is greater than zero, then migrate this context instance customer to monitoring context Customer in V' . Actually, from the new solution perspective, migrating the context instance from the existing solution can be considered as creating a new context instance. Therefore, each key metric much have one element mapping policy to set the value. Other than key metrics, non-key metric mapping policies may also be required for BPM solution evolution. It should be noted that if $V'.C.\lambda$ is a key metrics in the new solution, then all the operands must be static (i.e., the values are not updated after the value is assigned).

There are some constraints when migrating the context instance from an existing BPM solution to a new one.

1. If the context instances are direct descendants of the root context in the existing BPM solution, then the context that it migrated to in new BPM solution should also be the direct descendant of the root context.
2. If both context instances and the context that it migrated to in the new BPM solution are not direct descendants of the root context, then the context instances' parent context should also be migrated. Further, assume that context $V.C_1$ maps to context $V'.C'_1$, if context $V.C_1$'s parent context of $V.C_2$ maps to $V'.C'_2$, then $V'.C'_1$'s parent context is $V'.C'_2$.

¹in the rest of this paper, we use V to denote existing BPM solution, while V' denotes new deployed BPM solution.

4 Policy-driven BPM Solution Evolution

In this section, we first briefly describe our hybrid compilation-interpretation approach for implementing the BPM runtime engine, which is the foundation to implementing a policy-driven BPM solution evolution. We then give details on how the evolution policy is executed.

4.1 Foundation: Hybrid Approach for Observation Model Execution

In our previous work [10], we adopted a hybrid compilation and interpretation approach to realize the execution of observation models. We distinguish three kinds of application logic in a BPM solution: *common logic*, *information logic* and *model specific logic*. The common logic is defined in the meta-model level, which is applied to any observation model. One example of common logic is the routine for processing events. The information logic includes information organization in the observation model. Model specific logic (e.g., map expression) is unique to each individual model. The system consists of three components, namely model transformer, model compiler and model interpreter(see Figure 3). At buildtime, the model transformer decomposes the observation model into information logic and model specific logic. Also, it transforms the context-oriented model into an event-oriented model, which allows a more efficient runtime access to the model information. The compiler generates the object code for model specific logic. It should be noted that we adopted J2EE as our runtime platform. The object code of our model compiler is Java source code, which will be further compiled into Java library for runtime execution. Finally, the interpreter implements the common logic and interprets the information logic. It dynamically loads the model specific Java library to realize the operation of the observation model. Such an approach takes advantage of both compiling and interpreting. By adopting a compiling approach, our approach improves execution performance by generating customized code for individual models, while interpreting a model makes it easy to realize the model evolution because the interpreter maintains all the model information.

4.2 Design Rationale

If we consider evolution policies as a declarative program, like the observation model, there are also two approaches to executing policies, namely interpreting and compiling. We discuss adopting the interpreting approach first. The advantage of interpreting policies is that it is easy to realize the evolution policy execution because the policy interpreter can be easily integrated with the BPM runtime

engine as the runtime engine maintains all the model information. However, the logic of selecting context instances and mapping expression can be very complicated. Therefore, developing an interpreter that can execute map expressions optimally becomes difficult, given that the operators in constructing map expressions can be relational, set, vector, scalar, etc. Further, the elements referenced in mapping expressions are not limited to the same context. To locate the operands in mapping expressions, the interpreter needs to navigate through the hierarchy of contexts at runtime, which may also incur performance penalties. Such performance penalties may greatly impact the on-going situation detection.

Adopting a compiling approach can improve evolution policy execution performance by generating customized code for individual expressions, however generated code is a stand alone component, which is impractical to integrate with the BPM runtime engine. Such an approach can support hotswap-based evolution, however, it fails to support transition-based evolution. For example, in a transition-based evolution, some context instances need to phase out transactionally. Without integrating with the observation runtime engine, context instances may be terminated the when the situation detection is still in progress.

In order to take the advantage of our hybrid framework, we also compile and interpret evolution policies (See Figure 3). In our design, the *policy transfer* transforms evolution policies and stores data in refactored evolution policies, the *policy compiler* generates the runtime library for context instance selection predicate and mapping expressions. We also extend our model interpreter in the runtime engine to consume information in refactored evolution policies and dynamically load the evolution library for solution evolution.

4.3 Policy Transformation

One of the key steps in our policy-driven framework is to transform policies. Further, in order to facilitate compilation, the policy transformer pre-processes mapping expressions. As we discussed earlier, the design of the evolution policy language aims to facilitate the creation of policies. However, the organization of the data is not suitable for efficient runtime access. We design the policy transformer so it reorganizes the policies information (see data schema in Figure 4). In the data schema, a table is created for each type of elements in the evolution policies, including *ContextEvolution* for sunset and migration policies, *EventHandling* for event handling policies, *MetricEvolution* and *SituationEvolution* for mapping expression. Primary and foreign keys are used to represent the cross reference among the elements. For example, by foreign key *mappingID* in ta-

ble *MetricEvolution*, the metric evolutions that are belonging to a context evolution can be located.

Other than re-organizes information in evolution policies, the policy transformer also pre-processes policies. In particular, it collects mapping expressions that should be executed during the context evolutions and result is stored in the field of expressions in table *ContextEvolution*. Further, the policy transformer also pre-processes mapping expressions by determination of the navigation paths for each operand. Details about computing navigating paths can be found in our previous publication [10].

4.4 Runtime State Validation

In our framework, context migration consists of two basic steps: (1) Migrating the context instances and their key metric values; (2) Migrating the non-key metrics. Accordingly, there are two aspects of runtime state validation:

1. Context Instance Duplication Test. As we discussed earlier, mapping a context instance from one BPM solution to new one can be considered as creating a new context instance in new solution. Therefore, context instance duplication test is required. In our solution, a customized code is generated to facilitate the efficient runtime state validation. Before getting into details about code generation, we first briefly discuss the design of the runtime data store schema which the generated code runs on.

We designed a unified data schema (see Figure 5) that can be used by any observation model. To improve performance, our data schema stores the runtime objects vertically which minimizes the read/write conflicts. In our solution, there are four tables in the schema. One table *ContextInstances* (C) is used to store the context instances, while another table *Values* (V) is used to store the data of contexts. It should be noted that the content of the contexts are stored vertically in table *Values*. In table *Values*, each elementary element in contexts has a record in the table and *contextInstanceID* is unique for each context instance. Using *contextInstanceID*, the records in the table can be correlated to individual context instances. Table *Dimensions* (D) is used to store the dimension information when there exists array type of data elements in contexts. By specifying *dimensionOrder* and *sequenceID*, the datastore can store any dimension array of data in a context. Further, the table *Types* (T) gives type information.

Given the above unified datastore, we present how to generate queries to perform a duplication test. Assuming the target context instance's type is μ . The goal is try to search for the context instance that has same key metric values as created from the key-metric mapping expression.

The first step is generating queries to retrieve the context

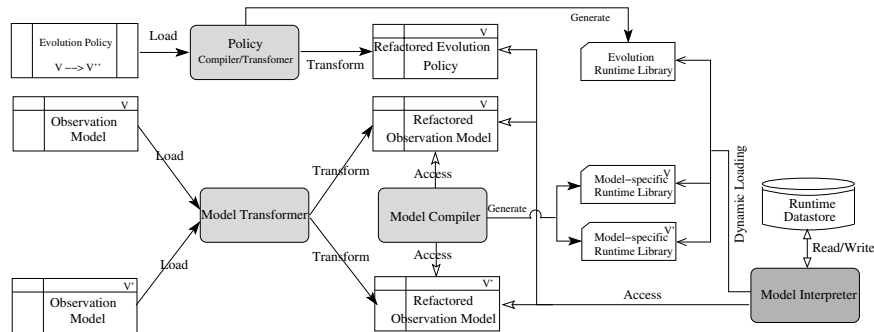


Figure 3. Hybrid Approach for Observation Model Execution and Solution Evolution

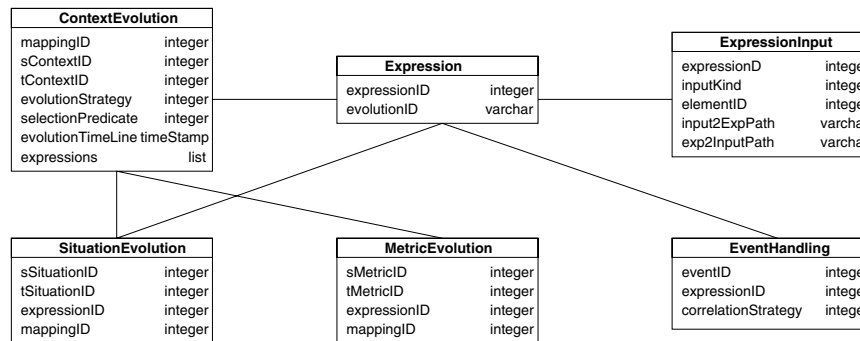


Figure 4. Simplified Data Schema for Evolution Policy

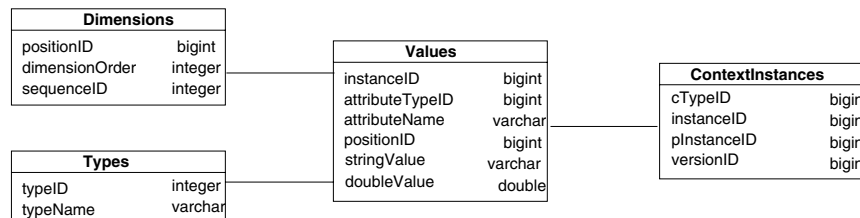


Figure 5. Runtime Datastore Data Schema

instances that belong to the migrating context type.

$$\rho(p, \pi_{instanceID}(C_{contextID=\mu})) \quad (2)$$

In the next step, we illustrate how to retrieve its key metric values for every attribute and position (if metric or attribute is single/multiple dimension array). Assuming there is a list of key metrics ($\lambda_1, \lambda_2, \dots, \lambda_m$) for the monitor context, and each metric has a list of attributes ($a_1, a_2, \dots, a_n$). If the a_j 's data type is string, then the query generated for retrieving $\lambda_i.a_j$ is:

$$\rho(v_{i,j}, \pi_{stringValue, positionID}(\sigma_{V.itemName=\lambda_i.a_j}(V \bowtie_{instanceID} p))) \quad (3)$$

This query joins the table `Values` with the context instances that have the same context type as the migrated context instance. Also assuming attribute a_j have l dimension array values and size of each dimension is d_k ($1 < k < l$), the extra queries that retrieving $\lambda_i.a_j$ in dimension $[x_1, x_2, \dots, x_l]$ ($0 < x_k < d_k$) is generated as:

$$\rho(d_{i,j,k}, (\sigma_{D.dimensionOrder=k \wedge D.sequenceID=x_k} D)) k \in [1..l] \quad (4)$$

For example, if a_j 's dimension size is 2 and the position needed to be retrieved is $[3, 5]$, then two queries are generated as it is a two dimension array: $\rho(d_{i,j,1}, (\sigma_{D.dimensionOrder=1 \wedge D.sequenceID=3} D))$ and $\rho(d_{i,j,2}, (\sigma_{D.dimensionOrder=2 \wedge D.sequenceID=5} D))$.

By equijoining $d_{i,j,k}$ on `positionID` to select dimension information on same tuple, we have query:

$$\rho(d_{i,j}[x_1, x_2, \dots, x_l], \bowtie_{positionID} \{d_{i,j,k}, k \in [1..l]\}) \quad (5)$$

It should be noted that each above query is according to a position in dimension k . Using the above example, by equijoining $d_{i,j,1}$ and $d_{i,j,2}$ we can have $\rho(d_{i,j}[3, 5], d_{i,j,1} \bowtie_{positionID} d_{i,j,2})$. In next step, by equijoining $v_{i,j}$ with $d_{i,j}$ using `positionID` to searching the fields, we have a query:

$$\rho(v'_{i,j}[x_1, x_2, \dots, x_l], (v_{i,j} \bowtie_{positionID} d_{i,j}[x_1, x_2, \dots, x_l])) \quad (6)$$

where $v'_{i,j}[x_1, x_2, \dots, x_l]$ represents the value of a key metric's attribute value in position $[x_1, x_2, \dots, x_l]$. Assuming the key-metric value in position $[x_1, x_2, \dots, x_l]$ that computed from mapping expression is $\lambda[x_1, x_2, \dots, x_l]$. The query for testing whether these two values are equal is:

$$\pi_{instanceID}(\rho(c_{i,j}[x_1, x_2, \dots, x_l], \pi_{stringValue}(v'_{i,j}[x_1, x_2, \dots, x_l]_{stringValue=\lambda[x_1, x_2, \dots, x_l]}))) \quad (7)$$

Therefore, by equijoining the `instanceID` for every position of every attribute in every key metric, we have the following query to test whether the context instance exists in the target solution.

$$\rho(c, \pi_{instanceID}(\bowtie_{instanceID} \{c_{i,j}[x_1, x_2, \dots, x_l]\} \mid i \in [1..m], j \in [1..n], x_k \in [0..d_k], k \in [1..l])) \quad (8)$$

In the runtime, generated queries are executed when the execution of all the key metric mapping policies are completed, in order to check whether a new context instance needs to be created on the target BPM solution.

2. Validating Freshness of Operands in Non-key Metric Mapping Policies. In second step of migrating context instance, non-key metric mapping policies are executed. The operands may be updated in on going event process (in the existing BPM solution) that is in parallel with the migration process. If that is the case, then the metric value that was computed based on the mapping expression in the migrated context instance is out of date. In such a situation, the execution of executing the non-key metrics mapping policies need to wait until the event processing is completed.

In order to efficiently determine the freshness of operand values in mapping policies, we analyze the observation model of the existing BPM solution to determine the metrics or situations that may be updated during the processing of the event (See Algorithm 1). The basic idea of the algorithm is to use the event as a trigger to initiate the expression computation (line 8) and then recursively use the output of expression to trigger other expressions (line 23). It should be noted that triggering a map computation can be either an event (line 10) or value change of a metric (line 14) or occurrence of a situation (line 18). By accumulating the output of the triggering expression (line 21), the list of elements that maybe updated during the processing of the event is obtained.

In the runtime, each context instance maintains a list of events (i.e., `Event [] currentProcessingEvents`) that are being processed. When executing the non-key metric mapping policy, if any operand's hosting the context instance's `currentProcessingEvents` is not empty and the operand is in `updateElements()` of any event in `currentProcessingEvents`, then the execution of the policy needs to be paused until the event processing is completed. At the same time, the runtime engine also queues events that may update the metric value until the execution of the mapping policy is completed (in both existing and the new BPM solution). Therefore, we can not only guarantee the freshness of the metric value, but also completeness of the event processing.

4.5 Executing Evolution Policy

In our approach, in order to transactionally shutdown the existing BPM solution, executing the sunset policies and event processing are integrated into model interpreter. The detail of the procedure of policy execution is:

1. Executing sunset policy.

- Executing context termination policy. The model

Algorithm 1: updateElements(Event e)

Data:

MapSet maps; a set that all the map expression in observation model

Event e; an event in observation model

Result:

updateElementSet; a set that contains all the metrics or situation that may be updated during processing event e

```
1 begin
2   updateElementSet  $\leftarrow \emptyset$ 
3   inputElementSet  $\leftarrow \{e\}$ 
4   processedElementSet  $\leftarrow \emptyset$ 
5   while inputElementSet is not empty do
6     get and remove an element from
      inputElementSet
7     processedElementSet += element
8     if element is an Event then
9       for map  $\in$  maps do
10        if event is trigger input of map then
11          process(updateElementSet, inputElements,
12                map.output)
13        else if element is a Metric then
14          for map  $\in$  maps do
15            if value change of the metric is trigger
16              input of map then
17                process(updateElementSet, inputElements,
18                      map.output)
19            else if element is a Situation then
20              for map  $\in$  maps do
21                if situation is trigger input of map then
22                  process(updateElementSet, inputElements,
23                        map.output)
24  Procedure process(Set updateElementSet, Set
25    inputElements, Element mapOutput) {
26    updateElementSet += {mapOutput}
27    if mapOutput is not in processedElementSet then
28      inputElementSet += {mapOutput}
29  }
```

interpreter loads and executes the runtime classes that were generated from the context instance selection predicates in the sunset policy. The selected context instances are terminated according to the time stamp given by the policy.

- Executing event handling policy. Execution of event handling policies is integrated with the event correlation process. When a new event is received, the model interpreter loads and executes the runtime classes that were generated from the event instance selection predicates to determine whether to ignore, or correlate to the existing context instance or create a new context instance.

2. Executing context migration policy. The model interpreter loads and executes the runtime classes that generated from context instance selection predicates in the migration policies. For the selected context instance, the model interpreter first executes the key-metric mapping expression to compute the key metric value. Then it executes the generated code for the duplication test. If a duplicated instance is found, the founded context instance is then the target instance for non-key metric migration. If the migrated instance is not a duplication, a new context instance is created in the target solution using the key metric values, and then non-key metric migration is performed.

5 Related Work

In this section, we review some work in software evolution, policy-driven frameworks and model-driven architecture.

Software evolution [7] is the dominant activities in software lifecycle. Currently, most of the work focuses on buildtime. In [5], the map metaphor is proposed, which can be served as an organizing principle for the design of effective tools for performing global software changes. Work in [9, 4] focus on metrics that measure the evolution between the releases of a software system. Instead of the buildtime aspect, our work tackles the runtime issues, i.e., how to support software evolution in the runtime systematically.

The policy-driven framework is recognized as an appropriate approach for managing distributed systems [8, 1, 11]. The policy-driven approach has the advantages of enabling the automated management and facilitating the dynamic behaviors of a large scale distributed system. In [8], a framework called *law-governed interaction* that provides a coordination and control mechanism for a heterogeneous distributed system. Policy works in standard bodies such as [1] focus more on defining frameworks for traditional IT

systems. In [11], a policy-driven exception-management framework for composite Web services is proposed. Instead of managing system behaviors within a solution lifecycle, our policy-driven framework focuses on the evolutions between the solution lifecycles.

Currently, model-driven architecture [6] is gaining momentum for enterprise application development. The current state of this practice employs the Unified Modeling Language (UML) as the primary modeling notation. The UML allows development teams to capture a variety of important characteristics of a system in corresponding models. Transformation among these models can be either manual or systematic. In our solution, UML is adopted as a modeling notation for defining the meta-model of an observation model. Based on the meta-model, a GUI based modeling tool is developed to allow solution providers to visually define the observation model. In regards to code generation, the Eclipse Model Framework [2] (EMF) enables code generation from UML models, wherein UML models have sufficient details about persistent and non-persistent data, business logic and presentation elements. However, in all cases, the models are the primary artifact created and manipulated by application developers, which is focused on IT implementation. In our solution, we adopt some principles of MDA, however MDA is focused on implementing a lifecycle of the software system, while our work focuses on evolution from one lifecycle to another.

6 Conclusions

In this paper, we present the design and implementation of dynamic evolution of BPM. We propose an evolution policy language to describe the mechanism of the BPM solution evolution. We extend our hybrid compilation-interpretation framework to execute evolution policies. First, based on evolution policy language, we developed a model transformer that refactors the evolution policies, which aims for facilitating the efficient execution of policies. Second, at transformation time, we derive relationships among the event processing progress and runtime state based on the observation model. Such relationship information not only allows efficient verification of the freshness of the migrating data, but also enables concurrent context instance migration and event processing. Third, we extend the runtime time infrastructure to interpret evolution policies, which allows for fine grained controls on solution evolution and supports execution of wide spectrums of evolution policies. Currently, we have implemented the whole system on J2EE platform. There are several on-going customer engagement efforts to verify our solution. Our future work includes scalability and reliability study of the system.

References

- [1] The IETF Policy Framework Working Group, 2006. <http://www.ietf.org/html.charters/OLD/policy-charter.html>.
- [2] Eclipse model framework, 2005. <http://fullmoon.torolab.ibm.com/tools/emf/scripts/home.php>.
- [3] Joachim H. Frank and Ghaly Gamil Stefanos. Business Operations Metamodel, Internal Design Document, IBM, January 2005.
- [4] Michael W. Godfrey and Qiang Tu. Evolution in open source software: A case study. In *ICSM*, pages 131–142, 2000.
- [5] William G. Griswold, Jimmy J. Yuan, and Yoshikiyo Kato. Exploiting the map metaphor in a tool for software evolution. In *International Conference on Software Engineering*, pages 265–274, 2001.
- [6] Model-driven architecture, 2005. <http://www.omg.org/mda/>.
- [7] T. Mens, J. Buckley, M. Zenger, and A. Rashid. Towards a taxonomy of software evolution. In *Towards a Taxonomy of Software Evolution, USE 2003*, 2003.
- [8] Naftaly H. Minsky and Victoria Ungureanu. Law-governed interaction: a coordination and control mechanism for heterogeneous distributed systems. *ACM Transactions on Software Engineering and Methodology*, 9(3):273–305, 2000.
- [9] Lehman MM, Perry DE, Ramil JF, Turski WM, and Wernick PD. Metrics and laws of software evolution—the nineties view. In *Metrics 97 Symp.*, Nov. 5-7th, 1997, Albuquerque, NM, 1997.
- [10] Liangzhao Zeng, Hui Lei, Michael Dikun, Henry Chang, and Kumar Bhaskaran. Model-driven business performance management. In *The 2005 IEEE International Conference on e-Business Engineering (ICEBE 2005)*, pages 32–39, Beijin, China, 2005. IEEE Computer Society.
- [11] Liangzhao Zeng, Hui Lei, Jun-Jang Jeng, Jen-Yao Chung, and Boualem Benattallah. Policy-driven exception-management for composite web services. In *7th IEEE International Conference on E-Commerce Technology (CEC 2005)*, 19-22 July 2005, München, Germany, pages 355–363. IEEE Computer Society, 2005.